MULTIMEDIA UNIVERSITY

# CCP6224 Object Oriented Analysis and Design
# Lab 11 – UML to code

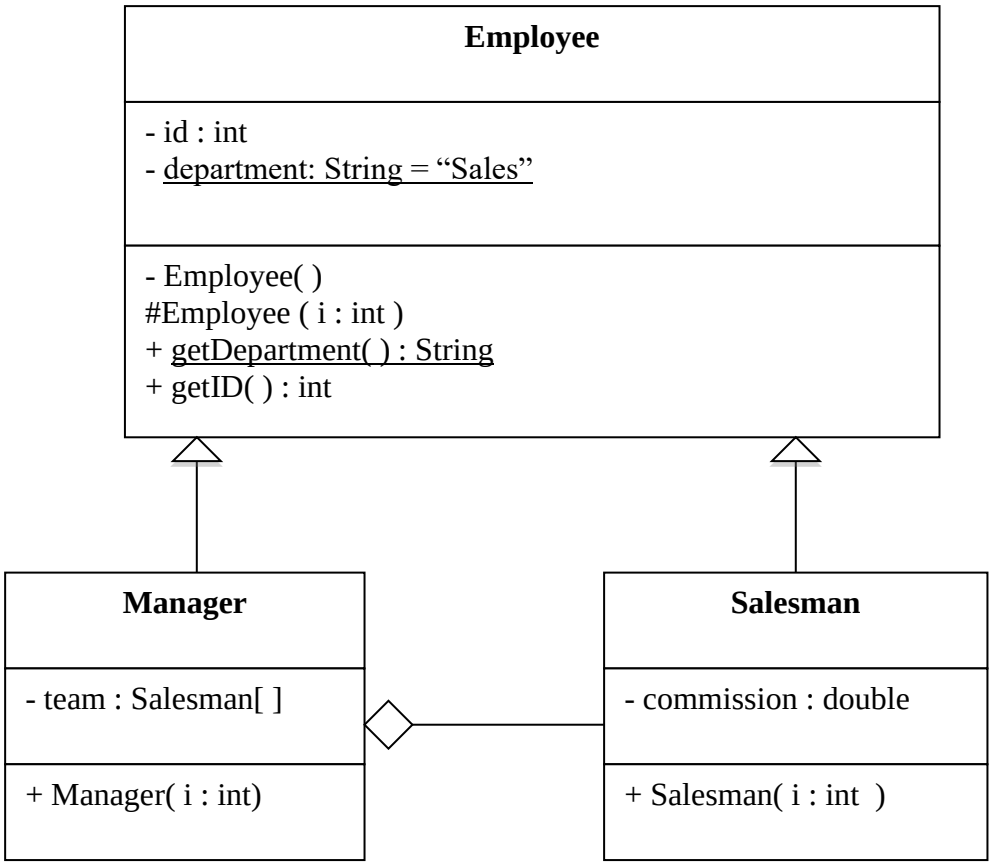
## Lab outcomes

By the end of today's lab, you should be able to

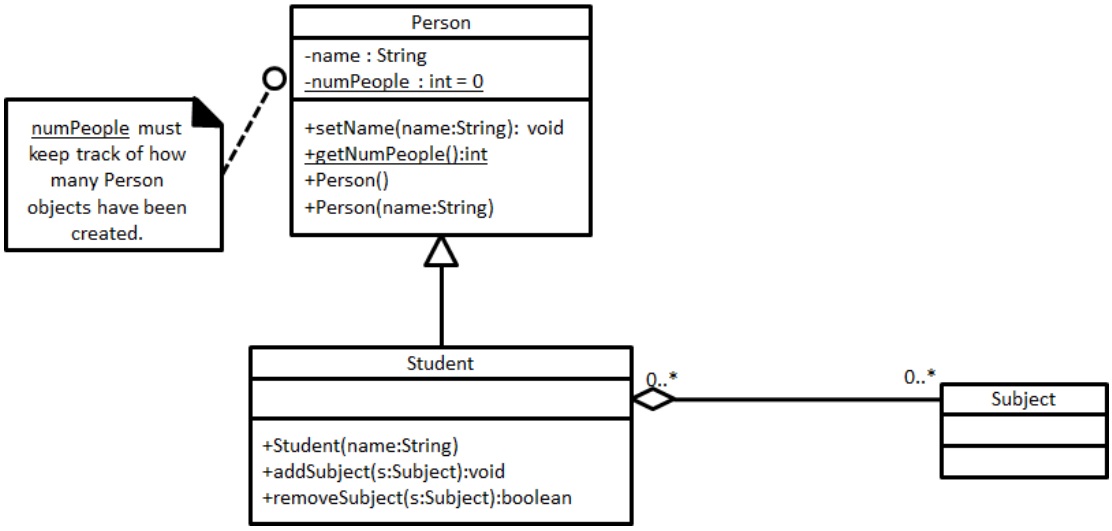
- translate between UML class diagrams and Java code and vice-versa

## UML to code

1. Write the Java program for class *Employee* and *Manager*, as shown in the UML class diagram below. Also fill in the bodies of the methods.



2. You are given the class diagram below for reference in the questions that follow




Write the Java code for the **`Person`** class. You must write the body of the code for each method and constructor. The constructors should make sure that **`numPeople`** will keep track of the number of **`Person`** objects created. The constructor that takes a parameter should rely on the constructor without a parameter to keep track of **`numPeople`** and the **`setName()`** to set the name.

Write the Java code for the Student class. You must write the body of the code for each method and constructor. The collection of subjects must be correctly maintained. The constructor must work correctly with the constructors of the superclass and still initialize the collection correctly.

3. Translate the following Java code snippets into its equivalent UML class diagram. Ensure that the correct access specifiers are used during the conversion to support the encapsulation properties of OOP

```
public class Employee {
    private static String department = "R&D";
    private int empId;
    public Employee(int employeeId) {
        this.empId = employeeId;
    }
    public static String getEmployee(int emplId) {
        if (emplId == 1) {
            return "idiotechie";
        } else {
            return "Employee not found";
        }
    }
    public static String getDepartment() {
        return department;
    }
}
```

```
public class Car {
    private String carColor;
    private double carPrice = 0.0;
    public String getCarColor(String model) {
```

```
        return carColor;
    }

    public double getCarPrice(String model) {
        return carPrice;
    }
}
```

```
public class PC{
      int i = 3;
      int j = 5;
      String name = "myName";

      public void getName(){};
}

public class CC extends PC{
      int i=3;
      int j=3;
      String name = "myName";

      public void getName(){};
}
```

```
public class OwnedClass{
      // body here
}

public class OwnerClass{
      private OwnedClass associatedClass;
      public OwnerClass otherClass;
      // body here
}
```

```
public class User{
      private String email;
      public String getEmail(){return email;}
      public void setEmail(String add){email=add;}
}

public class Owner extends User{
      private int maxNum;
      public int getMaxNum(){return maxNum;}
}
```

4. The following Java code for a program is provided. Perform the code conversion and draw a UML class diagram based on the code.

```
public interface Interface1 {
  public void method1 ();
}

public class Class1 {
  protected String field1;
}

public class Class2 extends Class1 implements Interface1 {
  int field2;
  public void method1 () {}
}
```

5. Using a UML class diagram, draw the classes which show a parent-child relationship from the code shown below.

```
public class Inheritance{

    public static void main(String args[]) {
      Server server = new Tomcat();
      Tomcat tomcat = (Tomcat) server;
       tomcat.start();
       System.out.println( "Uptime of Server in nano: " +
                                                server.uptime());
       tomcat.stop();
    }
}

class Server{
    private long uptime;

    public void start(){
      uptime = System.nanoTime();
    }

    public void stop(){
      uptime = 0;
    }

    public long uptime(){
      return uptime;
    }
}

class Tomcat extends Server{

    @Override
    public void start(){
        super.start();
        // ...
    }

    @Override
    public void stop(){
        super.stop();
        // ...
 }
}
```

6. Using a UML class diagram, draw the classes from the code shown

```
public class Recruit{
	char gender;
	private String name;
	private String[2] phone;
	public Recruit(){}
	protected void setGender(char s){}
	protected char getGender(){}
	public String getName(){}
	public void setName(String n){}
}

public class Weapon{
	private String type;
	public Weapon(){}
	protected void shoot(){}
	protected void reload(){}
	public Boolean isEmpty(){}
}

public abstract class Vehicle{
	public abstract void start();
	public abstract void stop();
}

public class Tank extends Vehicle{
	private Weapon gun;
	private Soldier[2] personnel;
	private static int idNumber;
	public Tank(){}
	public void addTroops(Soldier cst){}
	public Soldier getTroops(){}
	@Override
	public void start(){}
	@Override
	public void stop(){}
}

public Soldier extends Recruit{
	private String id;
	private Soldier();
	public Soldier(String n){}
	public String getID(){}
	public void setID(String n){}
}
```

7. Convert the following Java code into a UML Class diagram, showing all the details for every class including privacy and methods.

```
public interface Abba
{   public void song(String s);
}
public class BeeGees
{
    protected int beats;

    public BeeGees(int beats)
    {   this.beats = beats;
    }

    public int beater(int tempo)
    {   return beats * tempo;
    }
}
public class Beatles extends BeeGees implements Abba
{   private String songName;

    public Beatles(int beats)
    {   super(beats);
    }

    @Override
    public void song(String songName)
    {   this.songName = songName;
    }

    public void songBeater()
    {   int i;

        for (i=0; i<beats; i++)
            System.out.println(songName);
    }
}
```